

Face Detection

- As you probably guessed face detection is a process of detecting faces on image and video feeds/streams.
- So, what are the challenges? Can't we just train an object detection model, like YOLO, to detect faces?
- Yes of course you can. Modern YOLO are more than capable for this.

Face Detection

- Is this it!?
- Maybe? You can download a pre-trained model for face detection and go wild.
- But okay, obviously there are problems.
 - Multi person tracking
 - Low/high-light scenarios
 - real-time processing on edge/embedded hardware
 - working with small faces (i.e. crowds)
 - Extreme viewpoints
 - Occlusion

Face detection. Pre-history

- Before neural networks computer vision was more about geometry and gradients.
- OpenCV still carries two algorithms from those days:
 - Haar Cascades
 - Local binary patterns

Face detection. Pre-history

- Haar Cascades work by detecting areas of the image with high contrast changes.
- LBP splits and image into a grid.
 - For each grid cell it creates a feature vector, sort of like histogram.
 - This vector is then processed by a classifier to determine if the region contains a face.

Face detection. Yolo

- Don't try to reinvent a wheel!
- Often times you can find a pre-trained model for your task.
- For example, for this demo I will use a pre-trained weight for Yolo-11n model.

Face detection. MediaPipe

- MediaPipe is a very useful library and not just for face detection.
- It uses BlazeFace model, which took an inspiration from MobileNet series.
- Variants used in MediaPipe:
 - Short-range version, for selfies
 - Long-range version, for crowds
 - Sparse version, for even faster inference

Landmarks

- You probably noticed that MediaPipe also gave us facial landmarks alongside a bounding box.
- So why do we need them?
- Important data for emotion recognition (makes detecting smiles easier :)) and face recognition.
- Also, can help you create fun face filters.
- More advanced techniques will give you a 3d reconstruction of the face.

Face detection. YuNet

- OpenCV returns.
- YuNet claims a millisecond-level inference

Beyond Face Detection

- Detecting a face on an image is a rather banal problem.
- The questions is, what are you going to do with the face.
- Some directions:
 1. Emotion Detection
 2. Facial Action Unit Detection / Muscle Movement Detection
 3. Gaze Direction Detection / Eye Movement
 4. Key point detection / 3D Head Reconstruction
 5. Facial Recognition. Knowing which face belongs to who (evil).

Something to consider

- Don't forget pitfalls of neural networks
 - Bias! Classica example is racial bias. If you decide to train your own face detection model, make sure that datasets you use are not biased towards specific group of people
 - They can overfit! Model might give amazing results on your validation datasets, but if someone tilts their head weird or closes their mouth when yawning the same model can just break.

Emotion Recognition

- Can be as simple as image classification.
 - Usual emotion for this task are:
 - Anger
 - Disgust
 - Happiness
 - Sadness
 - Surprise
 - Neutral
- Or you can approach this as a multifaceted problem.
 - PyFeat

Emotion Recognition

- Images and videos are not end-all, be-all.
- If you can make use of other data inputs.
 - Sound
 - Text input
 - EEG
 - Heart Rate
- Sensor fusion!

Emotion Recognition

For emotion recognition demo we will use Deepface

Facial Recognition

- This when we get into creepy territory.
- Recognizing whose face belong to who.
- Surveillance
- Convert face region into a feature vector.
- Calculate similarity
 - Cosine
 - Euclidean
 - Etc.

Lab

1. Use any of the approaches presented. Detect and color invert your face
2. Building on top Ex.1, create a more advanced face filter. You can try using models that give you facial landmarks.
3. Test each model on its ability to detect small faces. Find image of crowds or use your webcam.
4. Use OpenCV to progressively apply noise and blur to your image. Test Yolo and MediaPipe.